

Private Friends Memory & Life Journal

Product Requirements Document (PRD) + Technical Requirements Document (TRD)

Purpose: A private, web-first digital space where a group of friends can collectively preserve photos, videos, blogs, stories, events and memories.

Build approach: MVP first. Keep the architecture production-ready, but avoid unnecessary complexity and features that are not required by the core use case.

1. Product Vision

Core concept

Create a private digital home for a small group of friends. The product should feel more like a shared private journal/memory archive than a public social network.

Core flow

Create Space → Invite Friends → Add Memories → Write Stories/Blogs → Discuss/React → Browse Timeline → Preserve memories long-term.

Product principles

Private by default; simple publishing; fast media uploads; chronological memories; group ownership; clean UI; no public follower economy.

2. Problem

Problem statement

Friends accumulate photos, videos, messages and stories across WhatsApp, Instagram, Google Photos and scattered folders. Those services are optimized for messaging, public/social feeds, or storage—not for maintaining one shared, private history of a friend group.

Desired outcome

One persistent private space where the group can document its shared life and easily revisit it later.

3. Target Users

Primary

Friend groups, college groups, travel groups, close-knit communities and small private circles.

Secondary

Families or private clubs can be supported later, but the MVP should remain focused on friend groups.

4. MVP Scope

Authentication

Email/password authentication; secure session handling; logout; password reset. Social login can be added later.

Private Spaces

A user can create a private space, give it a name and cover image, and invite members.

Membership

Owner/admin/member roles. Invite by secure token/link or email. Owner can remove members.

Profiles

Name, username/display name, avatar and short bio.

Posts / Memories

Create a memory with text, photos and optionally videos. Author and timestamp are visible. Edit/delete own posts; owner/admin moderation where appropriate.

Blog / Story posts

Long-form rich-text entries with title, cover image, body, media and publish date.

Timeline

Chronological feed of memories and stories. Infinite scroll or pagination.

Albums

Create shared albums and add photos. Album visibility follows the private space.

Comments

Members can comment on memories and blog posts. Authors can delete their own comments; admins can moderate.

Reactions

Simple reactions such as Like/Heart. Keep the initial reaction system small.

Events

Create an event/moment with title, date, description and optional media. Events can appear in the timeline.

Media

Image upload with preview, validation, compression/optimization and secure storage. Video support should be included only if the chosen storage/deployment setup can handle it reliably.

Notifications

Basic in-app notifications for invites, comments and reactions. Email notifications are optional for MVP.

Search

Search posts/stories by title/text and optionally filter by member, date or album.

Settings

Profile settings, space settings, membership management, privacy controls and account deletion.

5. Explicitly Out of Scope for MVP

Do not build

Public profiles, public feeds, follower/following system, algorithmic recommendations, ads, marketplace, live streaming, complex chat, AI-generated posts, video editing, advanced analytics, native mobile apps, complicated subscription tiers.

Rule for Jules

Do not add features merely because they are common in social apps. Any feature outside this document should be proposed first rather than silently implemented.

6. User Roles & Permissions

Owner

Create/delete space; invite/remove members; edit space settings; manage roles; moderate content.

Admin

Invite/remove members if permitted; moderate content; manage albums/events.

Member

Create memories/stories; upload media; comment/react; create or contribute to albums/events according to settings.

Guest

No guest access in MVP unless explicitly enabled by the owner. Private means authenticated members only.

7. Main User Journeys

New user

Register → verify account if enabled → create/join space → complete profile → see private timeline.

Create space

Create Space → name + description + cover → generate invitation → invite friends → dashboard.

Create memory

New Memory → write text → attach images/videos → preview → publish → appears on timeline.

Create story

New Story → title → cover → rich text editor → media → save draft/publish.

Invite friend

Space Settings → Invite → generate secure invitation → share link/email → recipient accepts → membership created.

Browse

Home → timeline → filter/search → open memory → view media/comments/reactions.

Leave/delete

Member can leave a space. Owner can transfer ownership or delete the space after confirmation.

8. Required Pages / UI

Public

Landing page, login, register, forgot/reset password, invitation acceptance.

Authenticated

Dashboard, space timeline, memory detail, story detail/editor, albums, album detail, events, member list, notifications, profile, settings.

Space admin

Space settings, members/roles, invitation management, moderation/content management.

UI requirements

Responsive desktop/tablet/mobile web UI; accessible forms; clear empty/loading/error states; drag-and-drop image upload where practical; optimistic UI only where data consistency is safe.

9. Recommended Technology Stack

Frontend

Next.js + TypeScript + Tailwind CSS + a mature component system such as shadcn/ui.

Backend

Next.js server/API routes or a clearly separated Node.js/Express service. Choose one approach and keep it consistent.

Database

PostgreSQL. Use Prisma or Drizzle ORM.

Authentication

Auth.js/NextAuth or another established authentication solution.

Media storage

S3-compatible object storage such as Cloudflare R2 or AWS S3. Store files outside the relational database.

Image processing

Server-side optimization or an image CDN/service. Generate thumbnails for galleries.

Deployment

Vercel/Cloudflare for web plus managed PostgreSQL and object storage.

Observability

Sentry or equivalent error monitoring; structured server logs.

Principle

Prefer managed services over building custom infrastructure for authentication, storage, image processing and email.

10. High-Level Architecture

Flow

Browser → Web App → Auth → API/server → PostgreSQL; media uploads use signed URLs → object storage/CDN.

Security boundary

The server must verify the authenticated user and space membership on every private resource request. Never rely only on frontend route guards.

Media access

Private media should not be publicly enumerable. Use signed URLs or an equivalent access-controlled delivery mechanism.

Background jobs

Only introduce a queue if media processing, notifications or other asynchronous work actually requires it.

11. Core Database Model

User

id, email, password/auth provider reference, display name, username, avatar URL, bio, createdAt, updatedAt.

Space

id, name, description, cover URL, ownerId, createdAt, updatedAt.

Membership

id, spaceId, userId, role, status, joinedAt.

Invitation

id, spaceId, inviterId, token/hash, email optional, expiresAt, acceptedAt, createdAt.

Post

id, spaceId, authorId, type [MEMORY/STORY], title optional, content, coverMediaId optional, status [DRAFT/PUBLISHED], publishedAt, createdAt, updatedAt.

Media

id, spaceId, uploaderId, storageKey, type, mimeType, size, width, height, duration optional, thumbnailKey optional, createdAt.

PostMedia

postId, mediaId, sortOrder.

Album

id, spaceId, creatorId, name, description, coverMediaId, createdAt, updatedAt.

AlbumMedia

albumId, mediaId, addedBy, addedAt.

Comment

id, postId, authorId, body, createdAt, updatedAt.

Reaction

id, postId, userId, type, createdAt; unique constraint on user/post/type as appropriate.

Event

id, spaceId, creatorId, title, description, eventDate, coverMediaId optional, createdAt.

Notification

id, userId, type, actorId, entityType, entityId, readAt, createdAt.

12. API Requirements

Auth

POST /auth/register, POST /auth/login, POST /auth/logout, POST /auth/forgot-password, POST /auth/reset-password.

Spaces

POST /spaces, GET /spaces, GET /spaces/:id, PATCH /spaces/:id, DELETE /spaces/:id.

Members

GET /spaces/:id/members, PATCH /spaces/:id/members/:userId, DELETE /spaces/:id/members/:userId.

Invites

POST /spaces/:id/invites, POST /invites/:token/accept, DELETE /invites/:id.

Posts

GET /spaces/:id/posts, POST /spaces/:id/posts, GET /posts/:id, PATCH /posts/:id, DELETE /posts/:id.

Comments/reactions

GET/POST/DELETE comments; POST/DELETE reactions.

Media

POST /media/presign, POST /media/complete, DELETE /media/:id.

Albums/events

CRUD endpoints plus membership-aware media operations.

Search

GET /spaces/:id/search?q=&type;=&author;=&from;=&to;=.

API rule

Every private endpoint must enforce authentication, membership and resource ownership/permission checks.

13. Privacy & Security

Required

HTTPS; password hashing using a proven algorithm; secure cookies/session handling; CSRF protection where applicable; rate limiting on auth/invite endpoints; input validation; output escaping; file MIME/type and size validation; authorization checks; audit-friendly logs.

Media security

Do not trust file extensions. Validate MIME/type and content where practical. Restrict upload size. Use randomized storage keys.

Privacy

Private spaces and their media must not appear in search engines. Avoid indexing private routes. Do not expose object-storage bucket listings.

Account deletion

Provide a clear account deletion flow. Define whether authored content is deleted, anonymized or retained according to the product's chosen policy before implementation.

Legal

Before public launch, add Terms of Service, Privacy Policy, acceptable-use rules and a process for abuse/copyright complaints.

14. Media Requirements

Images

JPEG/PNG/WebP/AVIF as appropriate; maximum upload size must be configurable; generate optimized display variants/thumbnails.

Videos

Use a configurable size/duration limit. Do not implement custom video transcoding in the first iteration unless required.

Upload UX

Progress indicator, cancel/retry, thumbnail preview, validation errors, failed-upload recovery.

Storage rule

Database stores metadata and storage keys, not raw binary media.

15. Performance Requirements

Targets

Fast initial page load; paginated/infinite timeline; lazy-load media; responsive images; database indexes on spaceId, authorId, createdAt and common search/filter fields.

Scalability

Design so media storage and delivery can scale independently from the application server.

Caching

Use caching only where it does not risk exposing private content between spaces/users.

16. Testing Requirements

Unit tests

Permission logic, invitation token handling, validation, post/comment/reaction rules.

Integration tests

Authentication, private-space access, media upload flow, CRUD APIs.

E2E tests

Register → create space → invite → accept → create memory → upload media → comment → react → search.

Security tests

Attempt cross-space access, unauthorized edits/deletes, invalid invite reuse, malicious file uploads and rate-limit bypasses.

17. Deployment & Environment

Required services

Application hosting, PostgreSQL, object storage, email provider if email flows are enabled, monitoring.

Environment variables

DATABASE_URL, AUTH_SECRET, STORAGE credentials/bucket configuration, APP_URL, email credentials if used, monitoring DSN if used.

Deployment pipeline

GitHub → CI checks → build/test → deploy. Production secrets must never be committed.

Backups

Managed database backups plus a documented media retention/recovery strategy.

18. Monetization — Later, Not MVP

Potential model

Free private space with limits on storage or members; paid plans could increase storage, media limits or advanced archival features.

Important

Do not build billing before validating that groups repeatedly use the product and value long-term storage/privacy enough to pay.

19. Product Analytics

MVP events

Space created, invite sent, invite accepted, memory published, story published, media uploaded, album created, comment added, reaction added, member returned.

Privacy

Collect only analytics necessary to improve the product. Avoid unnecessary personal tracking.

Success signal

Repeated contributions and return visits from multiple members of the same private space are more meaningful than raw sign-up count.

20. MVP Acceptance Criteria

Must work

A new user can register; create a private space; invite a friend; friend can join; both can publish memories; upload/view media; write stories; comment/react; browse timeline; search; manage membership.

Must be private

A non-member cannot read private posts, metadata or media by manipulating URLs/API calls.

Must be recoverable

Errors do not silently lose uploads or posts; failed operations show clear feedback.

Must be maintainable

TypeScript types, migrations, environment documentation, tests for authorization and a clear README are required.

21. Jules Implementation Plan

Phase 0 — Planning

Inspect repository; create architecture decision record; confirm stack; define environment variables; create implementation checklist. Do not code features outside scope.

Phase 1 — Foundation

Project setup, database, ORM, authentication, layout, design system, environment configuration and CI.

Phase 2 — Private Spaces

Spaces, memberships, invitations, roles and authorization middleware/server checks.

Phase 3 — Memories & Media

Post model, media storage, signed uploads, image optimization, timeline and detail pages.

Phase 4 — Stories

Rich-text editor, drafts, publishing, cover image and story detail pages.

Phase 5 — Social Layer

Comments, reactions and basic notifications.

Phase 6 — Albums & Events

Shared albums, event records and timeline integration.

Phase 7 — Search & Settings

Search/filtering, profile, privacy, space settings and account deletion.

Phase 8 — Hardening

Tests, security review, performance, error states, accessibility, deployment and documentation.

22. Jules Rules

Rule 1

Read this document before implementation and maintain a checklist of acceptance criteria.

Rule 2

Do not invent product requirements. If a decision is necessary but unspecified, choose the simplest reversible option and document it.

Rule 3

Never bypass authorization for convenience.

Rule 4

Do not store private media in a publicly accessible bucket.

Rule 5

Do not commit secrets or real credentials.

Rule 6

Avoid premature microservices, queues, complex caching and custom infrastructure.

Rule 7

Every major feature needs loading, empty, success and error states.

Rule 8

Run tests/lint/type-check after meaningful changes and fix regressions before moving on.

Rule 9

Keep UI responsive and accessible.

Rule 10

Before adding anything outside this PRD, explain the reason, trade-off and ask for approval.

23. Recommended Repository Structure

Structure

apps/web/ (Next.js app); components/; features/; lib/; server/; db/; prisma-or-drizzle/; tests/; public/; docs/; .env.example; README.md.

Architecture rule

Organize by feature/domain where practical. Keep database, authorization and media access logic centralized rather than duplicated across pages.

24. Definition of Done

Feature complete

Code implemented; types pass; lint passes; relevant tests pass; authorization verified; responsive UI verified; error/loading/empty states implemented; documentation updated; environment variables documented.

Release ready

Production build succeeds, migrations are reproducible, secrets are externalized, backups are configured, private media access is verified, and critical E2E flows pass.

25. Final Product Statement

One sentence

A private, collaborative digital memory space for friends to preserve their shared life through photos, videos, stories, blogs, events and conversations.

MVP priority

Privacy + collaboration + memories + media + timeline. Everything else is secondary.